

AI Service Desk 21-Day Build Plan

AI Service Desk — 21-Day Build Plan

Realistic full-time (6–8 hrs/day) schedule. This is the ideal pace — fast enough to be impressive, slow enough that nothing gets rushed or cut.

Day	Focus	Deliverable by end of day
1	Setup	Folder structure, env config, MongoDB + Redis connections, Express server boots cleanly
2	User model + Auth service	User model, auth.service.ts, JWT util (access + refresh)
3	Auth middleware + routes	auth / rbac middleware, register/login/refresh routes tested via Postman, httpOnly cookie setup
4	Ticket + Comment models	Both schemas finalized, indexes added, CRUD routes/controllers for tickets
5	Ticket routes polish + validation	Zod validation on all endpoints, centralized error middleware
6	Frontend scaffold + auth pages	Next.js app structure, login/register pages wired to real API
7	Protected routes + layout	Auth context/hook, protected route wrapper, role-based layout shell
8	Ticket list + create form	Ticket list page, create-ticket form, React Query hooks wired end to end
9	Ticket detail page	Single ticket view, comment thread UI (static data still fine for now)
10	Role dashboards	Employee, support-engineer, admin dashboard views with role-gated data fetching
11	AI service — categorization + priority	Gemini wrapper, structured-output prompt design, tested standalone before wiring in

Day	Focus	Deliverable by end of day
12	AI wired into ticket creation	Category/priority auto-filled on ticket create, summary generation added
13	Suggested solutions + prompt refinement	Suggestion generation, real time spent tuning prompts — the project's core differentiator
14	Duplicate detection	Embeddings + cosine similarity, tested against a small seeded dataset of similar tickets
15	Redis integration	Cache AI results, session handling, rate limit the AI endpoint properly
16	Socket.IO	Event contract designed, real-time ticket status updates, live comment broadcasting
17	Testing	Jest/Supertest — auth flow, ticket CRUD, AI service parsing, middleware
18	Docker + Docker Compose	Dockerfiles for client/server, compose file, full stack running via <code>docker-compose up</code>
19	CI/CD + deployment	GitHub Actions running tests on push, deploy backend (Render/Railway) + frontend (Vercel)
20	Buffer / bug fixing	Full walkthrough as a new user in each role, fix whatever breaks
21	README + polish	Architecture diagram, decision rationale, setup instructions, seed demo data, short demo video/GIF

Why this pace over a compressed 15-day version

- AI prompt work (days 11–13) gets 3 full days instead of 2 — it's the actual core of the project and worth protecting.
- Day 20 is a real buffer, not a hope. Integration bugs between Socket.IO, Redis, and the AI service will show up, and having a day to catch them beats shipping broken.
- Nothing gets silently cut.

If you fall behind — cut in this order

1. GitHub Actions CI — nice to have, not core; deploy manually if needed.

2. Duplicate detection — drop to a simple keyword-overlap check instead of embeddings.
3. Redis — the app functions without it; note it as “next step” in the README if cut.
4. Socket.IO — degrade to polling (React Query refetch interval) rather than true real-time, and be honest about that trade-off if asked.

Don't cut, no matter what: auth/RBAC, AI categorization + summarization, tests, and a README that explains your decisions. A smaller project that's fully working and well-explained beats a larger one that's half-broken.